

kamikami.eu — code review

Reviewed: `app.js`, `ui.js`, `blog.js`, `home.js`, `state.js`, `markdown.js`, `lazyload.js`, `devotional.js`, `mobile-tray.js`, `warning.js`, `style.css`, and the page HTML.

Ordered by impact. Items marked **verified** were reproduced by running the code, not just read.

1. Real bugs

1.1 Markdown images with alt text are broken — verified

`markdown.js`: the link replacement (line 63) runs *before* the image replacement (line 67). The comment on line 65 even says images must come first. The link regex happily matches the `[alt](src)` portion of `![alt](src)`.

```
Input:  ![A cat](/media/cat.webp)
Output: !<a href="/media/cat.webp" target="_blank">A cat</a>
```

Images only render when the alt text is empty (`![ ](...)`), because `[^\\ ]+` requires at least one character.

Fix: move the whole image `html.replace(/!\\[...\\]/...)` block (lines 67–78) above the links line.

1.2 Ordered lists never get a wrapper — verified

`markdown.js`: the `<ul>` wrapping pass (line 85) runs before ordered lists are converted to `<li>` (line 88), so `1. foo` produces bare `<li>` elements with no parent list.

```
Input:  1. first
        2. second
Output: <li>first</li>
        <li>second</li>
```

Fix: convert both list types first, tagging ordered items so they can be wrapped separately:

```
js
```

```
// Ordered lists
html = html.replace(/^\\d+\\. (.+)$/gm, '<li data-ol>$1</li>');
// Unordered lists
html = html.replace(/^\\- (.+)$/gm, '<li>$1</li>');
// Wrap consecutive runs
html = html.replace(/^(<li data-ol>.+<\\li>\\n?)+/gm, '<ol>$&</ol>');
html = html.replace(/^(<li>.+<\\li>\\n?)+/gm, '<ul>$&</ul>');
```

1.3 Half of `mobile-tray.js`'s `init()` never executes

The script tag has no `defer`, so at parse time `document.readyState === 'loading'` and `init` is bound to `DOMContentLoaded`. When `init` then runs, it registers *another* `DOMContentLoaded` listener (line 360). Per the DOM spec, listeners added during the dispatch of an event are not invoked for that dispatch — so lines 361–378 are dead.

Consequences: the nav-item click hook that refreshes the tray's post list never attaches, and the metadata polling loop never starts.

Fix: delete the inner `document.addEventListener('DOMContentLoaded', ...)` wrapper and run its body directly inside `init()`.

1.4 The mobile tray has no navigation links

`setupTrayEventListeners` queries `.mobile-nav-item` (line 102), but the tray template it builds (lines 30–52) contains only a posts section and theme buttons. `.mobile-nav-item` appears nowhere in the generated HTML, so that loop iterates over nothing and there's no way to change pages from the tray.

Either add the nav links to the template or delete the handler. `warning.js` also whitelists `.mobile-nav-item` in two places for an element that doesn't exist.

1.5 `runDevotional` can permanently lock itself out

`devotional.js` lines 85–105 set `devotionalActive = true`, then return early in three places (missing hero element, missing lead paragraph, missing verse) without resetting the flag. Once any of those fires, the devotional can never run again for the rest of the session.

Fix: wrap the body in `try { ... } finally { devotionalActive = false; }`, or reset before each return.

1.6 Stray closing tag in the tray template

`mobile-tray.js` lines 50–51 close one more `<div>` than were opened inside `.mobile-tray-content`. Browsers silently repair it, but remove the extra `</div>`.

1.7 `lazyload.js`'s "already loaded" guard never matches

Line 42 compares `img.src === dataSrc`. `img.src` returns a fully-resolved absolute URL; `data-src` is relative — so the comparison is always false. Every `mouseenter` after the image has loaded constructs a fresh `new Image()` and re-assigns `src`.

Fix: check `img.classList.contains('loaded')`, or compare `img.getAttribute('src')`.

1.8 `renderBlogCards` targets an element that doesn't exist

It renders into `#blog-posts-list`, which appears nowhere in the HTML. The function early-returns every time it's called. See §3.1 — it's legacy code and should go.

1.9 The `beforeunload` prompt fires on ordinary refreshes

`warning.js` lines 42–47 call `preventDefault()` whenever `bypassWarning` is false, which means refreshing or closing the tab shows the browser's "Leave site?" dialog unless the last click happened to land on a whitelisted element within the previous second. If that's intentional, fine — but it's a rough experience, and the 1-second window makes it fire unpredictably.

2. Performance

2.1 The devotional downloads the entire New Testament to show one verse

`devotional.js` fetches `/blog/nt_verses_compact.json` in full, then `rawData.map(...)` clones every entry into an object — doubling the memory — and `getRandomShortVerse()` re-filters the whole array on each call. All to display a single sub-150-character verse.

This is almost certainly the largest single payload on the site. Best fix, in order of preference:

1. Pre-generate a `verses_short.json` containing a few hundred already-filtered short verses at build time.
2. Have the server return one random verse from an endpoint.
3. At minimum, apply the `text.length < 150` filter *inside* the map so the long verses aren't retained, and index the raw tuples directly instead of rebuilding them as objects.

2.2 Three overlapping lazy-load mechanisms

Every generated `<img>` gets:

- native `loading="lazy"` (in the markdown output),
- an `IntersectionObserver` with `rootMargin: 50px`,
- a `mouseenter` handler.

The observer fires as soon as the image approaches the viewport, which entirely defeats the hover mechanism — the hover path is dead weight. Native `loading="lazy"` alone probably

covers the need.

Also, line 22 constructs a new `IntersectionObserver` for every image. If you keep the observer, use one shared instance created once at module scope.

2.3 Observer and listener leak on content replacement

`blog.js` replaces `#blog-article-content`'s `innerHTML` on every post open and every back-navigation. The discarded nodes still carry `mouseenter` listeners and are still being observed — nothing calls `unobserve` or `disconnect`. This accumulates across a reading session. A single shared observer plus an explicit `unobserve` before replacing content resolves it.

2.4 `backdrop-filter: blur()` on every card

There are 28 `backdrop-filter` declarations in `style.css`, including one on `.blog-card` itself (line 930). With a full grid of cards that's a per-card compositor blur, which is one of the most expensive effects available — and on a card sitting on a flat panel background there's very little behind it to blur anyway.

Keep it on the header and sidebar where it earns its cost; on cards, swap for a semi-opaque `background`. This is the most likely cause of scroll jank on mobile.

2.5 The 30-second autosave interval is redundant

`state.js` line 251: `setInterval(saveAppState, STATE_AUTO_SAVE_INTERVAL)`. State is already saved on every interaction (`handleInteraction`) and on `beforeunload`. The interval adds a full `JSON.stringify` + `localStorage.setItem` + `console.log` every 30 seconds, forever, whether or not anything changed. Delete it.

2.6 A 100ms poll that runs forever

`devotional.js` lines 131–139 poll `isWarningCleared()` every 100ms with no timeout and no clear path if consent is never granted. Replace it with an event:

```
js
```

```
// warning.js – inside the accept handler, after isAccepted = true
document.dispatchEvent(new CustomEvent('warning:cleared'));
```

```
js
```

```
// devotional.js
document.addEventListener('warning:cleared', () => {
  setTimeout(runDevotional, 300);
}, { once: true });
```

There's a second poll in `mobile-tray.js` line 370 — it at least has a 5-second cap, though it currently sits inside the dead block from §1.3.

2.7 Character-by-character typing forces a reflow per character

`typeWriteAnimation` and `typeDeleteAnimation` mutate `textContent` on a recursive `setTimeout` every 15–20ms. For a 150-character verse that's roughly 300 layout passes over ~5 seconds on the home page. Not fatal, but `requestAnimationFrame` with a time-based character count would be smoother and cheaper.

Worth noting separately: the devotional overwrites `.home-lead` permanently, destroying the original tagline with no way to restore it.

2.8 Ten unbundled scripts, two of them blocking

Ten separate JS requests, each an IIFE. Since they're all IIFEs, concatenating them in dependency order is safe and mechanical, and would cut nine round trips. `warning.js` and `mobile-tray.js` also lack `defer`, so they block HTML parsing — add it (after fixing §1.3, which depends on the current `readyState` behaviour).

2.9 Motion accessibility

Twenty animated particle divs, plus fades throughout. For a site that opens with a flashing-lights epilepsy warning, there's a conspicuous absence of:

CSS

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

Also consider halving the particle count on mobile.

3. Duplication and dead code

3.1 Legacy function pairs

Each of these exists in both a "legacy" and a "lazy" form, with the legacy version unreachable from `app.js`:

Legacy (dead)	Live version	Location
<code>renderBlogButtons</code>	<code>renderBlogButtonsLazy</code>	<code>home.js</code> — ~85% identical, 84 lines
<code>openBlogPostFromHome</code>	<code>openBlogPostFromHomeLazy</code>	<code>home.js</code>
<code>fetchBlogPosts</code>	<code>fetchBlogPostMetadata</code>	<code>home.js</code> / <code>blog.js</code>
<code>openBlogPost</code>	<code>openBlogPostLazy</code>	<code>blog.js</code>
<code>renderBlogCards</code>	<code>renderBlogPostSelectorGrid</code>	<code>blog.js</code> — also targets a missing element (§1.8)

Deleting the dead column removes roughly a third of `home.js`.

`window.blogIntroduction` in `markdown.js` — a lorem ipsum placeholder object — appears to be unused as well.

3.2 Four copies of "show the blog intro grid"

The same six-step sequence (hide post view → show intro view → clear `.post-selector-item.active` → render grid → update back button → save state) is written out four times:

- `ui.js` lines 109–138
- `ui.js` lines 145–174
- `state.js` lines 171–194
- `blog.js` `showBlogIntro()`, lines 490–507

This is the same duplication that produced the double-animation bug. Collapse to one exported function and call it from all four sites — it will prevent the next bug of that shape.

3.3 `warning.js` whitelist duplicated verbatim

The 19-selector `e.target.closest(...)` chain appears twice, identically: lines 17–35 and 212–232. Hoist it:

```
js

const SAFE_SELECTORS = ['a', '#themeToggle', '.menu-toggle', /* ... */];
const isSafeTarget = (el) => el && SAFE_SELECTORS.some(s => el.closest(s));
```

3.4 Duplicate mobile detection and resize handling

`isMobileView()` is defined twice — `ui.js` line 411 (breakpoint hardcoded as `768`) and `mobile-tray.js` line 9 (as `MOBILE_BREAKPOINT`). Two separate debounced `resize` listeners

with different delays (200ms and 100ms) do overlapping work. Consolidate into one shared helper and one listener.

3.5 Triple preload listeners per mobile post item

`mobile-tray.js` lines 286–304 attach `touchstart`, `mouseenter`, and `mouseover` — all calling `preloadBlogPostContent`. `mouseover` bubbles from the child divs, so it fires repeatedly during a single hover. The 150ms debounce inside `preloadBlogPostContent` absorbs most of the damage, but keep only `touchstart` + `mouseenter`.

3.6 Duplicated CSS selectors

Fifteen selectors are declared more than once at top level (not in media queries): `.about-content`, `.about-hero`, `.about-short`, `.blog-article`, `.blog-btn`, `.blog-intro-view`, `.blog-meta`, `.blog-post-view`, `.blue-button`, `.features-list`, `.home-content`, `.home-layout-container`, `.mobile-photo-placeholder`, `.photo-placeholder`, `.sidebar-toggle`.

Some are probably deliberate overrides, but they make the cascade hard to reason about. Worth a pass to merge or clearly comment.

3.7 Roughly 40 globals on `window`

Every module exports its entire surface, including internals used nowhere else:

`typeDeleteAnimation`, `loadImageOnHover`, `setCookie`, `getCookie`, `isFirstVisit`, `wrapHomeContentInRectangle`, `renderBlogCards`, `getRandomShortVerse`. Trim each module's export block to what's genuinely called across file boundaries — it makes the real dependency graph visible.

3.8 22 `console` statements in production

Concentrated in `state.js` (8) and `blog.js` (6). Notable offenders: one log per lazy-loaded image, and one every 30 seconds from the autosave interval. Gate them:

```
js
const DEBUG = false;
const log = (...a) => DEBUG && console.log(...a);
```

4. Design smell worth revisiting

The two-flag state machine in `setupNavigation` — `hasRestoredBlogSession` and `wasReadingBlogPost` — implements behaviour where clicking **Blogs** once restores the post you were last reading, and clicking it *again* shows the post grid. That's surprising: the same button does two different things depending on invisible state, and there's no affordance telling the visitor which they'll get.

It's also what made the original animation bug hard to spot, since it produced a different code path from About than from Home.

Consider making it unconditional — **Blogs** always shows the grid — and surfacing "continue reading *[title]*" as an explicit card at the top of that grid. Same convenience, no hidden state, and roughly 60 lines of branching disappears.

Suggested order of work

1. §1.1 and §1.2 — markdown fixes. Small, and images are currently broken on every post.
2. §2.1 — the New Testament payload. Largest single performance win.
3. §2.4 — drop `backdrop-filter` from `.blog-card`. One line, big mobile improvement.
4. §1.3 and §1.4 — the mobile tray's dead code.
5. §2.5 and §2.6 — remove the interval and the poll.
6. §3.1 and §3.2 — delete legacy duplicates, unify the intro-grid logic.
7. §4 — the navigation redesign, if the surprising behaviour isn't intentional.